

Experiment 5

Stock-Price Prediction - Multiple Linear Regression

Using lagged market features, scikit-learn LinearRegression(), chronological validation, and regression diagnostics

Aim

Build a Multiple Linear Regression model to predict the next observed closing price from lagged prices, moving averages, volume, and sentiment. Evaluate predictive performance without using future information and investigate the major regression assumptions.

Learning Outcomes

- Create lagged and rolling time-series predictors without look-ahead bias.
- Use a chronological train-test split rather than a random split.
- Fit and interpret a Multiple Linear Regression model.
- Evaluate predictions using MAE, MSE, RMSE, R2 and Adjusted R2.
- Diagnose linearity, autocorrelation, heteroscedasticity, residual normality, multicollinearity, and unusual observations.
- Explain why accurate price-level prediction does not automatically imply profitable trading.

Activity 1: Problem Statement

A financial analytics team wants a transparent baseline model for estimating the next trading day's closing price. Develop an MLR model using only information available before or at the prediction date. This is an educational synthetic dataset and must not be treated as investment advice.

Dataset

Use the accompanying stock_price_mlr.csv file. The first 12 rows are shown below.

Date	Close_Lag1	Close_Lag2	Close_Lag3	MA5	Volume	Volume_MA5	Sentiment	Next_Close
2026-01-09	139.98	139.56	139.16	139.4	1024665	969928.0	-0.029	139.08
2026-01-12	139.08	139.98	139.56	139.72	967834	968008.0	0.476	140.83
2026-01-13	140.83	139.08	139.98	140.4	977669	990005.0	-0.12	142.57
2026-01-14	142.57	140.83	139.08	141.4	1119730	1015273.0	0.136	144.53
2026-01-15	144.53	142.57	140.83	142.53	1104050	1038790.0	0.601	145.66
2026-01-16	145.66	144.53	142.57	144.26	1061190	1046095.0	0.442	147.71
2026-01-19	147.71	145.66	144.53	146.09	1072375	1067003.0	0.258	149.96
2026-01-20	149.96	147.71	145.66	148.04	1030473	1077564.0	0.659	152.36
2026-01-21	152.36	149.96	147.71	149.8	974583	1048534.0	0.316	153.33
2026-01-22	153.33	152.36	149.96	150.93	946515	1017027.0	0.114	151.27
2026-01-23	151.27	153.33	152.36	152.09	980408	1000871.0	0.372	153.54
2026-01-26	153.54	151.27	153.33	153.03	865049	959406.0	-0.173	154.67

Variables

Variable	Role	Description
Date	Order	Trading date; not used directly as a predictor
Close_Lag1	X1	Previous trading day's closing price
Close_Lag2	X2	Closing price two trading days earlier
Close_Lag3	X3	Closing price three trading days earlier
MA5	X4	Five-day moving average available at prediction time
Volume	X5	Current trading volume
Volume_MA5	X6	Five-day moving average of volume
Sentiment	X7	Synthetic market-news sentiment score (-1 to 1)
Next_Close	Y	Closing price to be predicted

Important: Real stock-price series are non-stationary and highly autocorrelated. This lab treats MLR as an interpretable baseline. Later work should compare returns, naive forecasts, and time-series models.

Step 1: Load and Inspect the Data

```
import pandas as pd
df = pd.read_csv("stock_price_mlr.csv", parse_dates=["Date"])
df = df.sort_values("Date").reset_index(drop=True)
print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())
```

Paste the output / graph below:

1. How many observations and predictors are present?
2. Are the dates sorted chronologically?
3. Are there missing values?
4. Which variable is the dependent variable?

Step 2: Descriptive Statistics and Exploratory Analysis

```
import matplotlib.pyplot as plt
import seaborn as sns
print(df.describe(numeric_only=True))
plt.plot(df["Date"], df["Next_Close"])
plt.xlabel("Date"); plt.ylabel("Next closing price")
plt.title("Next Closing Price Over Time")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

Paste the output / graph below:

Step 3: Correlation Analysis

```
corr = df.drop(columns=["Date"]).corr()
print(corr["Next_Close"].sort_values(ascending=False))
plt.figure(figsize=(9,7))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.tight_layout(); plt.show()
```

Paste the output / graph below:

5. Which predictor has the strongest relationship with Next_Close?

6. Which lag and moving-average variables are strongly related to one another?
7. Why can high predictor-predictor correlation make coefficient interpretation unstable?

Step 4: Define X and y and Make a Chronological Split

```
features = ["Close_Lag1", "Close_Lag2", "Close_Lag3", "MA5",
            "Volume", "Volume_MA5", "Sentiment"]
X = df[features]
y = df["Next_Close"]

split_index = int(len(df) * 0.80)
X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]
dates_test = df["Date"].iloc[split_index:]
print("Training observations:", len(X_train))
print("Testing observations:", len(X_test))
```

Why not `train_test_split(..., shuffle=True)`? Random shuffling can train the model on later dates and test it on earlier dates, producing information leakage and an unrealistic forecast evaluation.

Paste the output / graph below:

Step 5: Build the MLR Model

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print("Intercept:", model.intercept_)
for feature, coef in zip(features, model.coef_):
    print(feature, coef)
```

Paste the output / graph below:

Interpretation task: explain one coefficient as the expected change in `Next_Close` for a one-unit increase in that predictor while holding the other predictors constant. Discuss why highly correlated lags make this interpretation difficult.

Step 6: Predict and Compare

```
import numpy as np
y_pred = model.predict(X_test)
comparison = pd.DataFrame({
    "Date": dates_test.values,
    "Actual": y_test.values,
    "Predicted": y_pred,
    "Residual": y_test.values - y_pred
})
print(comparison)
plt.plot(comparison["Date"], comparison["Actual"], label="Actual", marker="o")
plt.plot(comparison["Date"], comparison["Predicted"], label="Predicted", marker="o")
plt.legend(); plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

Paste the output / graph below:

Step 7: Evaluate the Model and a Naive Baseline

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
n, p = X_test.shape
adjusted_r2 = 1 - (1-r2)*(n-1)/(n-p-1)

naive_pred = X_test["Close_Lag1"].to_numpy()
naive_rmse = np.sqrt(mean_squared_error(y_test, naive_pred))
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse)
print("R2:", r2, "Adjusted R2:", adjusted_r2)
print("Naive RMSE:", naive_rmse)
```

Paste the output / graph below:

Metric	Your value	Interpretation
MAE		
MSE		
RMSE		
R2		
Adjusted R2		
Naive RMSE		

A high R2 can occur because prices are persistent. The MLR model should therefore be compared with the naive baseline that predicts tomorrow's close as today's close.

Step 8: Create Training and Test Residuals

```
train_pred = model.predict(X_train)
train_residuals = y_train.to_numpy() - train_pred
fitted_values = y_pred
residuals = y_test.to_numpy() - y_pred
print("Training residual mean:", np.mean(train_residuals))
print("Test residual mean:", np.mean(residuals))
```

Paste the output / graph below:

Activity 2: Investigating Multiple Linear Regression Assumptions

Assumption 1 - Linearity

The conditional mean of the response should be adequately represented by a linear combination of the predictors. Inspect residuals against fitted values. Random scatter around zero supports the linear form; curves or repeated patterns suggest misspecification.

```
plt.scatter(fitted_values, residuals)
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted Values")
plt.show()
```

Paste the output / graph below:

Finding	Your observation	Conclusion
Residual pattern		
Linearity		Satisfied / Potential violation

Assumption 2 - Independence of Errors

Because these observations are chronologically ordered, independence is a major concern. Plot residuals in date order and use Durbin-Watson as a first-order autocorrelation diagnostic. A forecasting model that leaves serial structure in its errors may be missing time dependence.

```
from statsmodels.stats.stattools import durbin_watson
dw = durbin_watson(residuals)
print("Durbin-Watson:", dw)

plt.plot(np.arange(len(residuals)), residuals, marker="o")
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Observation order")
plt.ylabel("Residual")
plt.title("Residuals by Observation Order")
plt.show()
```

Paste the output / graph below:

Interpretation guide: a Durbin-Watson value near 2 suggests little first-order autocorrelation. Values below 2 suggest positive autocorrelation; values above 2 suggest negative autocorrelation. Interpret the value together with the sampling or time-order context.

Assumption 3 - Homoscedasticity

Residual variance should remain approximately constant across fitted values. A funnel or widening band suggests heteroscedasticity.

```
plt.scatter(fitted_values, residuals)
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Homoscedasticity Check")
plt.show()

# Optional Breusch-Pagan diagnostic (use training residuals for adequate sample size)
from statsmodels.stats.diagnostic import het_breuschpagan
X_bp = np.column_stack([np.ones(len(X_train)), X_train])
bp = het_breuschpagan(train_residuals, X_bp)
print("LM statistic:", bp[0])
print("LM p-value:", bp[1])
print("F statistic:", bp[2])
print("F p-value:", bp[3])
```

Paste the output / graph below:

Decision rule: describe the residual plot first. A small p-value (commonly < 0.05) supports evidence of non-constant variance, but the conclusion should not rely on the p-value alone.

Assumption 4 - Approximate Normality of Residuals

Normality is mainly relevant to classical small-sample inference. It is not a requirement that the raw response variable itself be normally distributed.

```
import scipy.stats as stats
plt.hist(residuals, bins=10, edgecolor="black")
plt.xlabel("Residual"); plt.ylabel("Frequency")
plt.title("Residual Distribution"); plt.show()

stats.probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals"); plt.show()

# Optional Shapiro-Wilk diagnostic
from scipy.stats import shapiro
stat, p_value = shapiro(residuals)
print("Statistic:", stat)
print("p-value:", p_value)
```

Paste the output / graph below:

Task: Describe the histogram and Q-Q plot first, then use Shapiro-Wilk only as supporting evidence. With a small test set, the plot and subject-matter context are especially important.

Assumption 5 - No Severe Multicollinearity

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
X_vif = X_train.astype(float)
vif = pd.DataFrame({
    "Feature": X_vif.columns,
    "VIF": [variance_inflation_factor(X_vif.values, i)
           for i in range(X_vif.shape[1])]
})
print(vif)
```

Paste the output / graph below:

VIF	Typical interpretation
Approximately 1	Little/no collinearity
1-5	Often acceptable
5-10	Potential concern

VIF	Typical interpretation
>10	Serious concern

Use these as rules of thumb. Interpret VIF alongside the correlation matrix, coefficient stability, and domain knowledge.

Assumption 6 - Outliers and Influential Observations

```

standardized_residuals = (
    residuals - np.mean(residuals)
) / np.std(residuals, ddof=1)

plt.scatter(fitted_values, standardized_residuals)
plt.axhline(0, color="black", linestyle="--")
plt.axhline(2, color="red", linestyle="--")
plt.axhline(-2, color="red", linestyle="--")
plt.xlabel("Fitted Values")
plt.ylabel("Standardized Residual")
plt.title("Standardized Residuals")
plt.show()

```

Paste the output / graph below:

Task: Identify observations with $|\text{standardized residual}| > 2$ and investigate them. A large residual is not automatically influential and must not be deleted without a defensible data or modelling reason.

Activity 3: Predict a New Observation

Feature	Value
Close_Lag1	158.4
Close_Lag2	157.85
Close_Lag3	157.1
MA5	157.62
Volume	1025000
Volume_MA5	985000
Sentiment	0.25

```

new_day = pd.DataFrame({
    "Close_Lag1": [158.40], "Close_Lag2": [157.85],
    "Close_Lag3": [157.10], "MA5": [157.62],
    "Volume": [1025000], "Volume_MA5": [985000],
    "Sentiment": [0.25]
})
new_day = new_day[features]
prediction = model.predict(new_day)
print("Predicted next closing price:", prediction[0])

```

Predicted next closing price:

Activity 4: Model Improvement Challenge

Build at least one alternative model. Options: remove redundant lag variables, predict the next-day return instead of the price level, add a trend term, use walk-forward validation, or compare Ridge regression. Do not select a model using the final test period repeatedly.

Discuss the strategy adopted for the alternate improved model you have developed.

Model	Predictors/change	R2	Adjusted R2	RMSE	Naive RMSE	Conclusion
Model 1	Original					
Model 2	Improved					

Final Assumption Checklist

Assumption	Diagnostic	Evidence/result	Conclusion
Linearity	Residual vs fitted		
Independence	Design/context + residual order/DW		
Homoscedasticity	Residual plot + optional BP		
Residual normality	Histogram + Q-Q + optional Shapiro		
No severe multicollinearity	Correlation + VIF		
No problematic outliers	Standardized residuals + investigation		

Submission Requirements

- Completed activity sheet with pasted outputs and graphs.
- Executable Python notebook or .py file.
- A one-page evidence-based diagnostic report.
- Original-versus-improved model comparison.